

Applied Machine Learning

Training Neural Networks



UNIVERSITY OF
GOTHENBURG

CHALMERS

Richard Johansson

`richard.johansson@cse.gu.se`

training feedforward neural networks

- ▶ training a NN consists of finding the weights in the layers
- ▶ so how do we find those weights?

training feedforward neural networks

- ▶ training a NN consists of finding the weights in the layers
- ▶ so how do we find those weights?
- ▶ as we did for the SVC and LR!
 - ▶ state an **objective function** including a **loss** and possibly a **regularizer**
 - ▶ apply an **optimization algorithm** to find the weights that minimize the objective

—

SGD: pseudocode

initialize $\mathbf{w}$

repeat ...

 pick a training instance $(\mathbf{x}_i, y_i)$

 compute gradient ∇f_i of the loss for current instance $(\mathbf{x}_i, y_i)$

$$\mathbf{w} = \mathbf{w} - \eta \cdot \nabla f_i(\mathbf{w})$$

return $\mathbf{w}$

loss functions

- ▶ **log loss** for a **binary classifier**

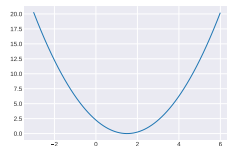
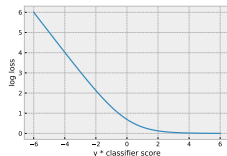
$$\text{Loss} = -\log(P(\text{output} = y))$$

- ▶ **cross-entropy loss** for **multiclass**

$$\text{Loss} = -\log(P(\text{output} = y))$$

- ▶ **squared error loss** for **regression**

$$\text{Loss} = (y - o)^2$$



example

- ▶ let's use two layers with sigmoid units, and then the log loss

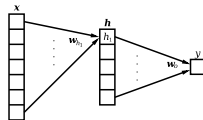
$$\mathbf{h} = \sigma(\mathbf{W}_h \cdot \mathbf{x})$$

$$y = \sigma(\mathbf{W}_o \cdot \mathbf{h})$$

$$\text{Loss} = -\log(y)$$

- ▶ so the whole thing becomes

$$\text{Loss} = -\log \sigma(\mathbf{W}_o \cdot \sigma(\mathbf{W}_h \cdot \mathbf{x}))$$



- ▶ now, to do gradient descent, we need to compute gradients w.r.t. the weights $\mathbf{W}_h$ and $\mathbf{W}_o$

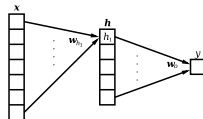
example

- ▶ let's use two layers with sigmoid units, and then the log loss

$$\mathbf{h} = \sigma(\mathbf{W}_h \cdot \mathbf{x})$$

$$y = \sigma(\mathbf{W}_o \cdot \mathbf{h})$$

$$\text{Loss} = -\log(y)$$



- ▶ so the whole thing becomes

$$\text{Loss} = -\log \sigma(\mathbf{W}_o \cdot \sigma(\mathbf{W}_h \cdot \mathbf{x}))$$

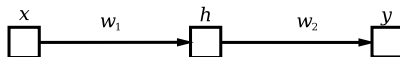
- ▶ now, to do gradient descent, we need to compute gradients w.r.t. the weights $\mathbf{W}_h$ and $\mathbf{W}_o$
- ▶ **ouch!** it looks completely unwieldy!

the chain rule of derivatives/gradients

- ▶ NNs consist of functions applied to the output of other functions
- ▶ in calculus, we use the **chain rule** to compute gradients of nested functions
 - ▶ assume that we apply the function f to the output of g
 - ▶ then the **chain rule** says how we can compute the gradient of the combination:

$$\text{gradient of } f(g(x)) = \text{gradient of } f(g) \cdot \text{gradient of } g(x)$$

chain rule example

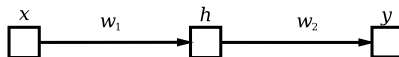


- ▶ let's say we have defined a simple neural network:

$$h = f_1(w_1 \cdot x)$$

$$\text{Loss} = f_2(w_2 \cdot h)$$

chain rule example



- ▶ let's say we have defined a simple neural network:

$$h = f_1(w_1 \cdot x)$$

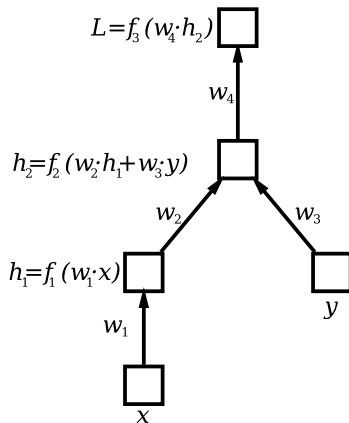
$$\text{Loss} = f_2(w_2 \cdot h)$$

- ▶ then we can compute the gradients with respect to w_1 and w_2 using the chain rule:

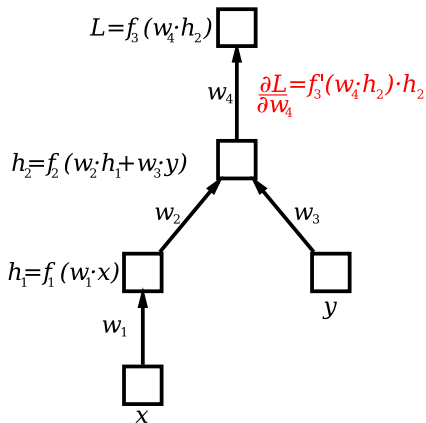
$$\frac{\partial \text{Loss}}{\partial w_2} = f'_2(w_2 \cdot h) \cdot h$$

$$\frac{\partial \text{Loss}}{\partial w_1} = f'_2(w_2 \cdot h) \cdot w_2 \cdot f'_1(w_1 \cdot x) \cdot x$$

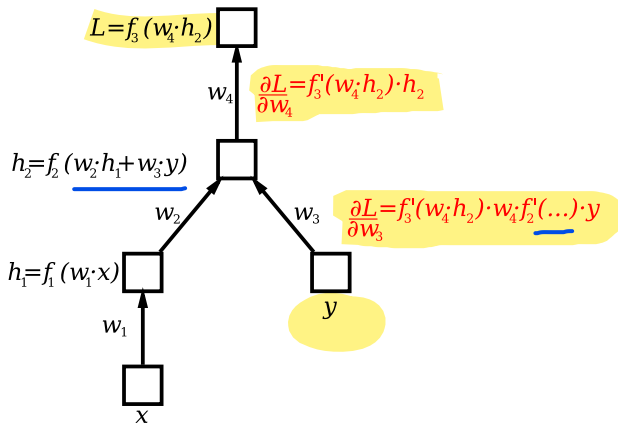
slightly more complicated computational graph



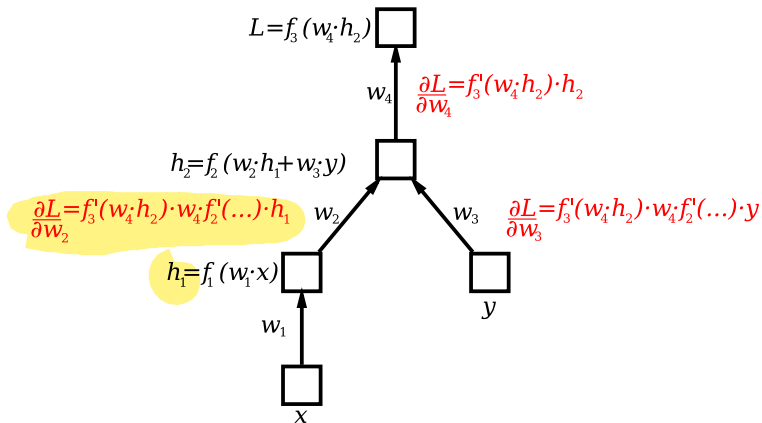
slightly more complicated computational graph



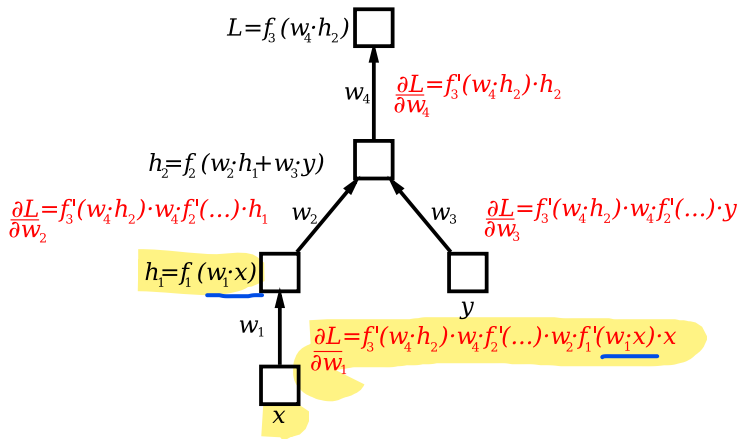
slightly more complicated computational graph



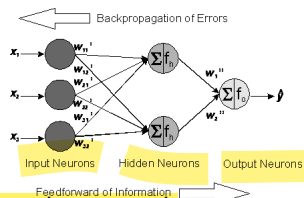
slightly more complicated computational graph



slightly more complicated computational graph



the general recipe: backpropagation



- ▶ using the chain rule, the gradients of the weights in each layer can be computed from the gradients of the layers after it
- ▶ this trick is called **backpropagation**
- ▶ the general form of backprop is a bit kludgy and involves a lot of matrix book-keeping
- ▶ fortunately, there are computer programs that can do the algebra for us!
 - ▶ in NN software, we usually just declare the network and the loss, then the gradients are computed under the hood

neural network training: summary

- ▶ the key idea is the same as for linear models: we define an **objective** and we set the weights to **optimize** it
 - ▶ e.g. the **cross-entropy** loss optimized with **SGD**
- ▶ optimization requires that we **compute gradients**
- ▶ gradients are computed by the **backpropagation** algorithm
 - ▶ backprop is based on the **chain rule**